

problem 3 explanation

goal

we need to check if bessie can visit at least V farms if she wakes up at time S .

for each farm i , it is only visitable if bessie arrives *before* it closes.

- arrival time: $t_i + S$
- closing time: c_i .

which we can deduct an inequality:

$$t_i + S < c_i$$

to make this easier to work with, we can rearrange the inequality to isolate S .

$$S < c_i - t_i$$

which tells us for any given wake up time S , bessie can visit farm i only if S is less than $c_i - t_i$; which is the farm's closing time minus the time it takes for bessie to travel to said farm. we shall call this difference d_i .

$$S < c_i - t_i = d_i \therefore S < d_i$$

based on the equation, for each query $Q_i = (S, V)$, what we really want to find is the amount of farms that have a d_i value that is greater than S .

solution one

this solution is straightforward. for each query, we loop through the amount of farms, and check if the farm closes before bessie can arrive.

```
# this is pseudocode
for (S, V) in Q:
    visitable_farms = 0
    for i from 1 to N:
        d_i = c[i] - t[i]
        if S < d_i:
            visitable_farms += 1
    if visitable_farms >= V:
        print("YES")
    else:
        print("NO")
```

this works, however, it is a little slow because we have to loop through all the farms for every single query – we can make it a bit better.

solution two

since the condition we need to check is just comparison $S < d_i$, we can pre-process the data. we can make a new array with d_i values, and let's call this `d_values`. for each farm i , we calculate $d_i = c_i - t_i$, and put it in `d_values`, and most importantly, we sort the array in *ascending* order.

then, we can answer the queries. for each query $Q_i = (S, V)$: we need to find how many numbers in the sorted `d_values` array are strictly greater than S .

since the array is sorted, we can use binary search to find this count pretty fast! we can use a function like `upper_bound` (in c++) or `bisect_right` (in python). it finds the index of the first element that is greater than S .

let's say `upper_bound` gives you an index j . this means all elements from `d_values[j]` to the end of the array are greater than S ; and the number of visitable farms is $N - j$.

now, just check if $N - j \geq V$. if it is, print YES, otherwise print NO.

it should look something like this:

```
# this is pseudocode

# precompute
d_values = []
for i in 0 to N-1:
    d_i = c[i] - t[i]
    d_values.append(d_i)

d_values.sort()

for (S, V) in Q:
    index = binary_search_for_first_greater(d_values, S) # first d_i > S
    visitable = N - index # count visitable farms
    if visitable >= V:
        print("YES")
    else:
        print("NO")
```